

The How-to of Programming the uSDX (and others)

WAY more than you ever wanted to know

Author's note: I didn't create any of this. This works for me. Try it and see if it works for you. We are Standing on the Toes of Giants. Thanks, as always, to [Rob Colclough](#) and [Muhammad Iqbal](#) and especially Manuel DL2MAN, Guido PE1NNZ and Barbaros WB2CBA for all of their efforts over the YEARS!

Please comment with corrections and advise!

THE ARDUINO UNO

The AI tells us: "The Arduino UNO is a beginner-friendly, open-source microcontroller board based on the [ATmega328P](#), commonly used for building digital devices, interactive projects, robotics, and IoT applications. It is programmed via the Arduino IDE using C++ and features 14 digital I/O pins, 6 analog inputs, and a 16 MHz ceramic resonator."

This tiny computer may be used with the Arduino IDE to program the ATmega328P in the uSDX as well as other ATMEL devices. The IDE software is freely available for Windows, Linux, and macOS. As an open source design, the UNO is available from many vendors, including Amazon, AliExpress and TEMU for very little money.

The UNO will be used with a special program to drive the standard ATMEL six-pin serial programming port identified as a 3x2 header.

Install the IDE

The Arduino IDE is available for Windows, Linux, and macOS. The environment works the same for any of these. The latest version is available at <https://www.arduino.cc/en/software/>

Download the installer for your system and install. The IDE includes an editor, the GCC compiler, and debugger for the ATMEL line of chips. Examples and extensive documentation is also available on the Arduino web site.

Attach an Arduino UNO

This document will focus on the UNO R3 and it's clones. This version attaches to a host with a USB cable.

Attach the UNO to the cable and then the cable to the computer. The device should be found and drivers installed in most cases.

Verify the IDE and UNO

Many example projects are installed with the IDE. The projects are called 'Sketches' and the C++ source code files in text format are indicated with a .INO extension.

Load 'Files → Examples → 01.Basics → Blink'. The 'Blink' Sketch is a single file which runs a simple loop, turning the LED on for a second, and then off for a second.

This app may be loaded any time to verify proper UNO connection and operation.

Configure the IDE

Select Files → Preferences and click on 'Select verbose output during' 'compile' and 'upload'.

Select 'verify code after upload'.

These options will provide a higher level of detail during operation.

Run the Blink Sketch

Select 'Sketch->Upload'. Review the output in the console window. Look for the LED blinking on the UNO board.

The console reports:

```
AVR device initialized and ready to accept instructions
Device signature = 1E 95 0F (ATmega328P, ATA6614Q, LGT8F328P)
Reading 924 bytes for flash from input file Blink.ino.hex
in 1 section [0, 0x39b]: 8 pages and 100 pad bytes
Writing 924 bytes to flash
Writing | ##### | 100% 0.18s
Reading | ##### | 100% 0.13s
924 bytes of flash verified
Avrdude done. Thank you.
```

Success

This is a simple process to verify the UNO is operational. It may be used at any time.

PROGRAM THE UNO FOR ISP

As stated earlier, the UNO board may be programmed to serve as the In-circuit Serial Programmer (ISP). In many cases, a uSDX radio will include a dual-row, six pin, 0.10 inch header.

The ISP header allows direct programming of the processor chip without requiring removal. Key connections include MOSI, MISO, SCK, RESET, and GND.

The ISP Sketch is called 'ArduinoISP'.

Select 'Files->Examples->11.ArduinoISP->ArduinoISP'

Select 'Sketch->Upload'

The console reports:

```
AVR device initialized and ready to accept instructions
Device signature = 1E 95 0F (ATmega328P, ATA6614Q, LGT8F328P)
Reading 4354 bytes for flash from input file ArduinoISP.ino.hex
in 1 section [0, 0x1101]: 35 pages and 126 pad bytes
Writing 4354 bytes to flash
Writing | ##### | 100% 0.73s
Reading | ##### | 100% 0.57s
4354 bytes of flash verified
Avrdude done. Thank you.
```

CABLE FOR ISP PROGRAMMING

Five jumper wires are used from the UNO to the ATMEL six pin ISP header. The UNO does NOT supply ENOUGH power to the target CPU of a full radio. Use your regular power supply.

There are several good YouTube videos available on connecting a UNO to a uSDX radio. I refer to 7 minutes into <https://youtu.be/PES8uGIUP8Y>

This is a good reference for me. Nice photos.

PIN #	Function	Color	ISP PIN
10	RESET	Orange	5
11	MOSI	Yellow	4
12	MISO	Green	1
13	SCK	Blue	3
GND	GND	Purple	6

PROGRAM uSDX v4.00e

The original release by Guido PE1NNZ may still be on GitHub:
<https://github.com/threeme3/usdx>

Version 1.02x is commonly the release loaded on the various uSDX radios when shipped. It has many shortcomings and issues. Memory is not particularly well managed, so optional modules are conditionally compiled using define statements. It is common to configure the file to create binaries which are too large to fit in the available target flash!

This code has been forked and revised several times. The current code is v4.00e. The maintainers have done extensive refactoring to allow all available options to be enabled in the 'e' release.

Releases are published in the 'Files' section of the Facebook group, '(tr)uSDX Open'.

Download the v4.00e.INO file for your radio:
<https://www.facebook.com/groups/219627530721541/files/files>

All of the 4.00e files are the same. [Muhammad Iqbal](#) has been gracious enough to make versions for most common configurations with the required edits.

Make a folder with the same name (i.e. 400e-WHITE-BUTTONS) and move the .INO file there.

Open/Double Click/Launch your INO file. Program the unchanged file as a baseline test.

Sidebar: There are many configuration options available in the .INO file. In particular, you may want to set your callsign and initials for use in automated CW calls. Make a copy of the hardware specific file (possibly name with your callsign), create a folder with the same name, move the .INO file into the new folder, make edits, program from this file.

Configure for programming. Select 'Tools' and configure 'Board' - 'Arduino UNO', 'Port' - set to the port where your UNO is found, and 'Programmer' - 'Arduino as ISP'.

Turn on the radio. The UNO does not supply power to the target CPU.

Select 'Sketch->Upload Using Programmer'.

PLEASE NOTE. This is DIFFERENT than just 'Upload'. The code is being loaded across the ISP to the target CPU – not the ISP CPU.

PLEASE NOTE. Actual programming takes a long time - in excess of 30 seconds. Anything less and there is likely an issue with the wiring from the UNO to the radio.

The console shows:

```
AVR device initialized and ready to accept instructions
Device signature = 1E 95 0F (ATmega328P, ATA6614Q, LGT8F328P)
Auto-erasing chip as flash memory needs programming (-U flash:w:...)
specify the -D option to disable this feature
Erased chip
Reading 31200 bytes for flash from input file 400e-WHITE-BUTTONS.ino.hex
in 1 section [0, 0x79df]: 244 pages and 32 pad bytes
Writing 31200 bytes to flash
Writing | ##### | 100% 34.07s
Reading | ##### | 100% 18.99s
31200 bytes of flash verified
Avrdude done. Thank you.
```

RESET THE uSDX

The uSDX configurations must be reset with each new release. There may be new variables and defaults may need to change.

Power off the radio. Remove the wires from the UNO to the radio.

Reset the configurations by holding down 'MENU' while turning on the power.

Make note of the text on the display. Mine says:

```
9M2GRC R4.00e  
Reset settings...
```

After releasing 'MENU', the radio boots normally.

Select 'MENU' and turn the knob to review the new menu items.

CUSTOMIZE THE FIRMWARE

Open the .INO file in the Arduino IDE. Please note, by the fine work by [Rob Colclough](#) and [Muhammad Iqbal](#), the v4.00e version has ALL optional components enabled by default.

There are five configurations in the v4.00e .INO file. Nobody can count the actual number of uSDX variants there really are.

Note that major configuration starts at line 155. You may comment out a define with '//' and remove the '//' to uncomment for a different configuration.

The MY_CALLSIGN define may be changed on line 189. Be sure to change MY_CALLSIGN_PADDED on line 291, and CALLSIGN_LENGTH on line 300.

FOOTNOTES

The uSDX class of SDR radios has many members. There are handhelds, 1, 3, 5, and 8 bands. Some devices use a 25, 26, or 27 MHz base frequency. There are various hardware configurations, including handhelds and variants with built in antenna tuning units. There are orange boxes, orange tops, and orange buttons.

The code remains the same for any variant. With a few tweaks, uSDX radios may be kept up to date with the latest software developments.